CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

MINI PROJECT REPORT

SUBJECT: - Python Programming(25CAH-606)

Project Title:- Sales Analysis Dashboard

Submitted By:

Name: Anurag Shukla
UID: 25MCI10308
CLASS: MCA(AI/ML)
SECTION: - 25MAM-5A

Submitted to:

Deepali Saini
Assistant Professor

University Institute of Computing Chandigarh University, Gharuan, Mohali

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

TABLE OF CONTENT

S.No	Topic	Page No.
1	Introduction	01
2	Background	01
3	Objective	02
4	Technology Used	02
5	Requirement	02
6	System Design	02
7	Code	03
8	Output	08
9	Conclusion	09
	Reference	10

1. Introduction

Games have always been an integral part of human entertainment, evolving from simple board games to sophisticated computer-based interactions. Among these, the Snake Game holds a nostalgic place, being one of the earliest games introduced on mobile phones and computers.

This project presents a simple yet engaging implementation of the Snake Game using the Python programming language and the Pygame library.

The project focuses on creating a 2D graphical interface where a snake moves around the screen, eats food, grows in length, and ends when it collides with walls or itself. The goal of the player is to keep the snake alive as long as possible and achieve the highest score.

2. Background

The Snake Game concept originated in the 1970s and became widely popular after being featured on Nokia mobile phones. The game's logic revolves around basic programming concepts such as loops, conditionals, and event handling.

Python, being a powerful high-level language, provides excellent support for game development through its Pygame module. Pygame is a cross-platform set of Python modules designed for writing video games, which includes computer graphics and sound libraries.

This project uses Pygame to develop an interactive, colorful, and fully functional version of the Snake Game, emphasizing both logic building and game design.

3. Objective

The main objectives of this project are:

- To design and implement a fully functional Snake Game using Python and Pygame.
- To demonstrate the concepts of event handling, loops, and collision detection.
- To enhance logical thinking and programming skills through practical application.
- To create a fun and interactive program that promotes user engagement.

  To provide a foundational understanding of how simple games can be created using Python libraries.

4. Technology Used

The technologies and tools used in this project are:

Component	Description
Programming Language	Python 3.10 or higher
Library Used	Pygame
IDE Used	Visual Studio Code
Operating System	Windows 10
Graphics	2D rectangle-based visuals (built-in Pygame rendering)

Why Pygame?

- It is beginner-friendly and easy to learn.
- Provides built-in modules for graphics, sound, and keyboard input.
- Highly portable and runs on all major operating systems.
- Ideal for creating 2D arcade-style games.

5. Requirement

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 100 MB free space
- Display: 1024 × 768 resolution

Software Requirements

- Python 3.10 or later
- Pygame Library (Install using pip install pygame)
- Visual Studio Code or any Python IDE
- Windows / macOS / Linux OS

6. System Design

The system design follows a simple modular approach consisting of the following components:

1. Initialization Module o Initializes the Pygame window, sets up display, and defines colors and fonts.
2. Game Loop Module
   - o Handles all events, updates the snake's position, and renders graphics continuously until the game ends.
3. Collision Detection o Checks if the snake collides with the boundary or with itself.
4. Food Generation o Randomly generates food items within the window.
5. Score Display o Continuously updates and displays the player's score.
6. Restart/Quit Functionality o Allows the player to restart or quit after losing.

Flow of Execution

Start → Initialize Game → Display Window
↓
Get User Input (Arrow Keys)
↓
Move Snake → Check Collision → Update Score
↓
If Collision → Display "Game Over" Message
↓
Restart or Quit
↓
End

7.Code

```
import pygame
import time
import random
pygame.init()

# Colors white = (255,
255, 255) yellow = (255,
255, 102) black = (0, 0,
0) red = (213, 50, 80)
green = (0, 255, 0) blue
= (50, 153, 213)

# Display size dis_width
= 800 dis_height = 600

# Create window dis = pygame.display.set_mode((dis_width,
dis_height)) pygame.display.set_caption(' Snake Game by
Anurag Shukla')
```

```
clock = pygame.time.Clock()
snake_block = 20 snake_speed
= 15

font_style = pygame.font.SysFont("bahnschrift", 25) score_font
= pygame.font.SysFont("comicsansms", 30)

def your_score(score):     value = score_font.render("Score: " +
str(score), True, yellow)     dis.blit(value, [10, 10])

def our_snake(snake_block, snake_list):
for x in snake_list:
      pygame.draw.rect(dis, green, [x[0], x[1], snake_block, snake_block])

def message(msg, color):
   mesg = font_style.render(msg, True, color)
dis.blit(mesg, [dis_width / 6, dis_height / 3])

def gameLoop():
game_over = False
game_close = False
x1 = dis_width / 2     y1
= dis_height / 2
x1_change = 0
y1_change = 0
snake_List = []
   Length_of_snake = 1

   foodx = round(random.randrange(0, dis_width - snake_block) / 20.0) * 20.0
foody = round(random.randrange(0, dis_height - snake_block) / 20.0) * 20.0
```

```
   while not game_over:
while game_close:
        dis.fill(blue)
        message("You Lost! Press C-Play Again or Q-Quit", red)
your_score(Length_of_snake - 1)          pygame.display.update()

        for event in pygame.event.get():
           if event.type == pygame.KEYDOWN:
if event.key == pygame.K_q:
                    game_over = True
game_close = False                  if
event.key == pygame.K_c:
                    gameLoop()

      for event in pygame.event.get():
if event.type == pygame.QUIT:
            game_over = True           if
event.type == pygame.KEYDOWN:
if event.key == pygame.K_LEFT:
x1_change = -snake_block
y1_change = 0              elif event.key ==
pygame.K_RIGHT:
                x1_change = snake_block
y1_change = 0              elif event.key ==
pygame.K_UP:                   y1_change = -
snake_block                x1_change = 0
elif event.key == pygame.K_DOWN:
```

```
                y1_change = snake_block
x1_change = 0

        if x1 >= dis_width or x1 < 0 or y1 >= dis_height or y1 < 0:
            game_close = True

        x1 += x1_change
        y1 += y1_change         dis.fill(black)         pygame.draw.rect(dis, red,
[foodx, foody, snake_block, snake_block])         snake_Head = [x1, y1]
snake_List.append(snake_Head)         if len(snake_List) >
Length_of_snake:
            del snake_List[0]

        for x in snake_List[:-1]:
if x == snake_Head:
game_close = True

        our_snake(snake_block, snake_List)
your_score(Length_of_snake - 1)         pygame.display.update()

        if x1 == foodx and y1 == foody:
            foodx = round(random.randrange(0, dis_width - snake_block) / 20.0) * 20.0
foody = round(random.randrange(0, dis_height - snake_block) / 20.0) * 20.0
Length_of_snake += 1

        clock.tick(snake_speed)

    pygame.quit()
    quit()

gameLoop()
```

8.Output



Fig 1 (Home Screen)

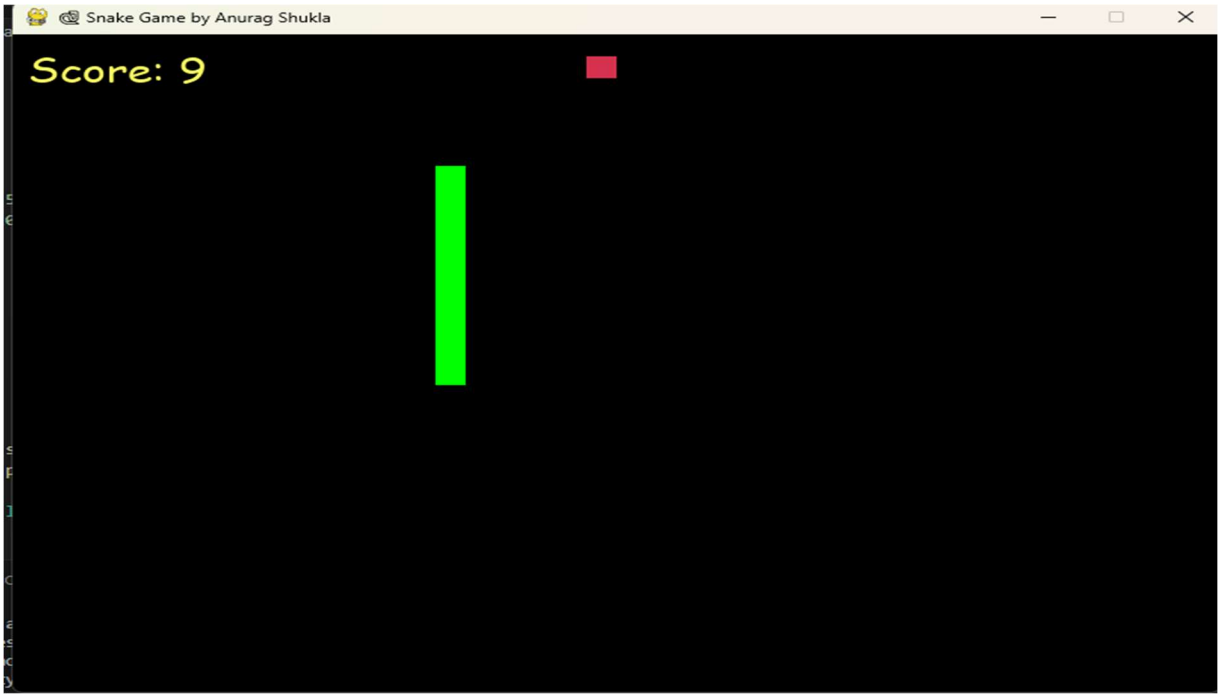


Fig 2 (Screen while playing the game)

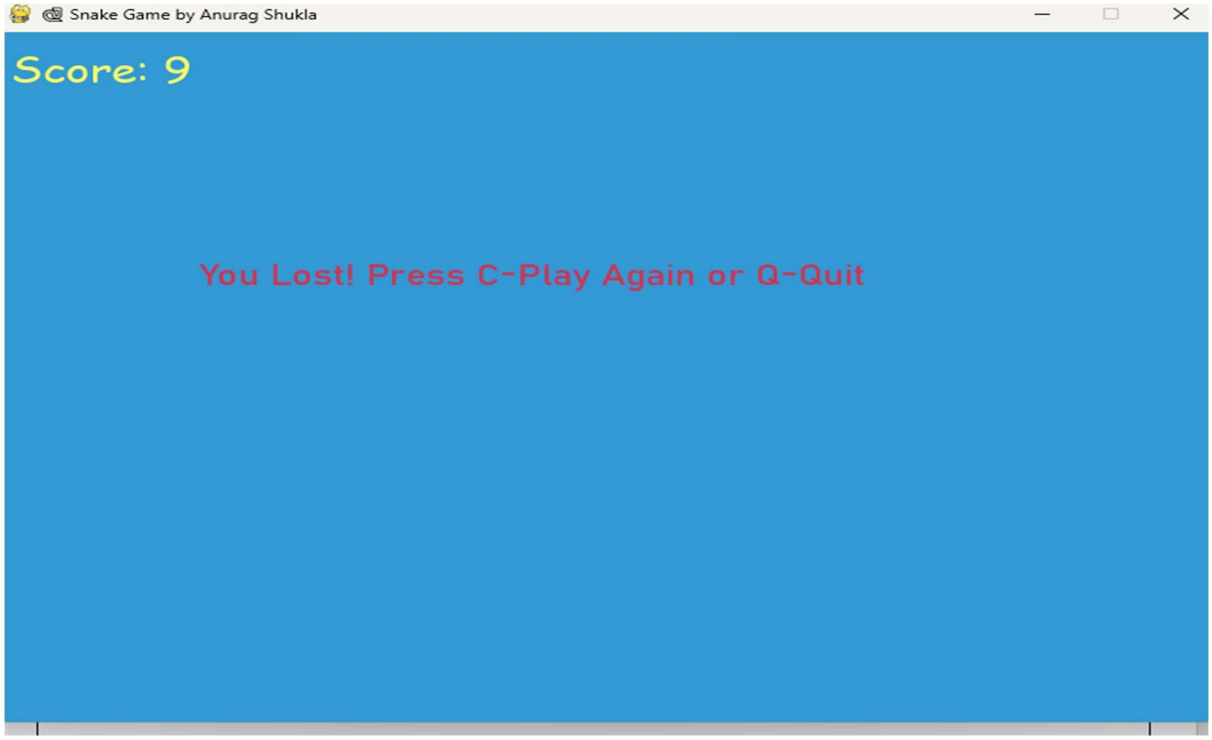


Fig 3 (Output Screen)

When the program is executed, a game window appears with the title “Snake Game by Anurag Shukla”.
The snake starts at the center of the window.
Gameplay

- The player moves the snake using arrow keys.
- Each time the snake eats food (red block), it grows longer.
- The score is displayed at the top-left corner.
- The background changes color dynamically during events.

Game Over Screen
When the snake collides with the wall or itself, a message is displayed:
“You Lost! Press C to Play Again or Q to Quit.”

9. Conclusion

The Snake Game Project successfully demonstrates the fundamental concepts of Python programming, game logic, and Pygame usage. Through this project, we learn about:

- Event-driven programming
- Keyboard input handling
- Collision detection
- Dynamic graphics rendering

This project is a strong foundation for creating more complex games in the future, such as maze solvers, shooting games, or car racing simulations.

References

1. https://github.com/Anuragshukla21/snake-game-by-anurag-shukla
2.Python Official Documentation – https://docs.python.org/3/
3.Pygame Official Site – https://www.pygame.org/
4.TutorialsPoint – Python Game Development Tutorials
5.GeeksforGeeks – Pygame Basics
6.W3Schools – Python Programming Guide